lab: 6 title: "Знакомство с Kubernetes (продолжение)" min_score: 5 max_score: 10

Лабораторная работа №6

Kubernetes на двух машинах и CI/CD: автоматическая сборка образа и выкатка в кластер

Максимум: 15 баллов.

Продолжаем ЛР №5 с тем же приложением (FastAPI, PostgreSQL, Redis) и теми же манифестами из `k8s/`. Теперь кластер будет из двух узлов, а образ приложения будет собирать и публиковать GitHub Actions.

Персональные параметры (заполните **ДО** начала работы):

Параметр	Как определить	Ваше значение
Номер ИСУ	Можно взять из ЛК	-
Персональный префикс	Фамилия латиницей в нижнем регистре (например, ivanov)	-

NodePort приложения	30000 + последние 2 цифры ИСУ Пример: 30000 + 45 = 30045	-
Секретный ключ	secret+последние 2 цифры ИСУ+фамилия на английском	-
Имя базы данных	фамилия+последние 2 цифры ИСУ	-
Напарник (ФИО, ИСУ)	Вторая машина кластера	-

> [!IMPORTANT] Все команды, имена узлов и манифесты должны использовать ваши персональные значения. Отчёты с NodePort по умолчанию или без префикса `<фамилия>` - автоматически получают 0 баллов за соответствующие этапы.

Цель работы

1. Собрать кластер k3s из двух машин в одной сети.
2. Настроить в GitHub Actions сборку образа и публикацию в GHCR.
3. Развернуть приложение из ЛР №5 так, чтобы поды были на обоих узлах.
4. Обновить приложение и посмотреть, как проходит rolling update.

Подготовка: кто с кем работает

Работаем парами (если у вас две ВМ, можно и одному). Один поднимает master (control plane), второй присоединяется как worker. Потом можно поменяться ролями.

Что подойдёт в качестве двух машин:

Вариант	Что нужно	Комментарий
Два ноутбука с Linux	Одна сеть (проводная или Wi-Fi)	Самый простой вариант
Две виртуальные машины	Режим сети Bridged (мост) на обоих хостах	NAT не подходит: узлы не увидят друг друга
Ноутбук + ВМ на ноутбуке напарника	Bridged для ВМ	Хост ВМ и вторая машина - в одной сети

Нужен Linux (Ubuntu 22.04 или 24.04 подойдут), 2 vCPU, от 2 ГБ памяти, `sudo` и интернет.

Проверка сети

Это нужно сделать до начала работы, на каждой машине:

```
ip -4 addr show | grep inet          # узнайте свой IP, нап
ping -c 3 <IP_напарника>             # должен отвечать
```

Между машинами должны проходить эти порты (если включён `ufw`, откройте их или отключите файрвол на время работы):

Порт	Протокол	Назначение
6443	TCP	API-сервер Kubernetes (worker → master)
8472	UDP	Flannel VXLAN (сеть подов между узлами)
10250	TCP	kubelet
30000-32767	TCP	NodePort-сервисы

Этап 1. Кластер из двух узлов

Задание 1.1 - Установите k3s server на машине №1 (master)

```
curl -sfL https://get.k3s.io | INSTALL_K3S_EXEC="server -
```

Проверьте:

```
sudo k3s kubectl get nodes -o wide
```

Получите токен для присоединения узла:

```
sudo cat /var/lib/rancher/k3s/server/node-token
```

Задание 1.2 - Присоедините машину №2 (worker)

```
curl -sfL https://get.k3s.io | K3S_URL=https://<IP_master
```

Задание 1.3 - Проверьте кластер (на master)

```
sudo k3s kubectl get nodes -o wide
```

Должны быть два узла в состоянии `Ready` с разными `INTERNAL-IP`.

Для удобства скопируйте kubeconfig и используйте обычный `kubectl`:

```
mkdir -p ~/.kube && sudo cp /etc/rancher/k3s/k3s.yaml ~/.
export KUBECONFIG=~/.kube/config
kubectl get nodes
```

Задание 1.4 - Пометьте узлы метками

```
kubectl label node <фамилия>-master role=db
kubectl label node <фамилия>-worker role=app
kubectl get nodes --show-labels
```

Вопрос: почему PostgreSQL с PersistentVolumeClaim приходится привязывать к одному узлу, если используется storage-класс `local-path`?

Этап 2. CI в GitHub Actions

Задание 2.1 - Подготовьте репозиторий

Откройте репозиторий с прошлых работ, создайте из `lab-5` ветку `lab-6` и убедитесь, что в корне лежат `Dockerfile` и каталог `k8s/` из ЛР №5. Репозиторий должен быть на GitHub и публичным.

Задание 2.2 - Создайте workflow `.github/workflows/ci.yml`

```
name: ci

on:
  push:
    branches: [main, lab-6]
  workflow_dispatch:

permissions:
  contents: read
  packages: write

env:
  IMAGE: ghcr.io/${{ github.repository }}
```

```yaml
jobs:
  test:
    runs-on: ubuntu-latest
    defaults:
      run:
        working-directory: solutions/assist-py
    steps:
      - uses: actions/checkout@v4
      - uses: actions/setup-python@v5
        with:
          python-version: "3.11"
      - name: Install dependencies
        run: pip install --no-cache-dir -r requirements.t
      - name: Syntax check
        run: python -m compileall app

  build-and-push:
    needs: test
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4
      - uses: docker/setup-qemu-action@v3
      - uses: docker/setup-buildx-action@v3
      - uses: docker/login-action@v3
        with:
          registry: ghcr.io
          username: ${{ github.actor }}
          password: ${{ secrets.GITHUB_TOKEN }}
      - id: meta
        uses: docker/metadata-action@v5
        with:
          images: ${{ env.IMAGE }}
          tags: |
            type=sha,format=short
            type=raw,value=latest
      - uses: docker/build-push-action@v6
        with:
          context: ./solutions/assist-py
          platforms: linux/amd64,linux/arm64
```

```
          push: true
          tags: ${{ steps.meta.outputs.tags }}
```

Пояснения:

- `needs: test` значит, что образ соберётся только после успешной проверки.
- В `platforms` две архитектуры, потому что у напарника может оказаться ARM (например, ВМ на Mac с Apple Silicon).
- `metadata-action` сам переводит имя образа в нижний регистр (GHCR этого требует) и добавляет тег с коротким SHA коммита.

Задание 2.3 - Запустите пайплайн

```
git add .github/workflows/ci.yml
git commit -m "lab6: add CI"
git push origin lab-6
```

Откройте вкладку Actions в репозитории и дождитесь, пока оба job станут зёлёными.

Задание 2.4 - Сделайте образ публичным

Зайдите в GitHub → профиль → Packages → ваш пакет → Package settings → Change visibility → Public. Потом проверьте, что образ скачивается без авторизации:

```
docker pull ghcr.io/<github_user>/<repo>:latest
```

Этап 3. Развёртывание на двух узлах

Задание 3.1 - Обновите манифесты

В `k8s/app-deployment.yaml`:

1. Укажите образ из GHCR: `image: ghcr.io/<github_user>/<repo>:<sha>`. Берите короткий SHA из Actions, тег `latest` не подходит.
2. Поставьте `replicas: 4`.
3. Чтобы поды разъехались по обоим узлам, добавьте в шаблон пода `topologySpreadConstraints`. `nodeSelector` для приложения не используем, иначе все поды уйдут на worker:

```
    spec:
      topologySpreadConstraints:
        - maxSkew: 1
          topologyKey: kubernetes.io/hostname
          whenUnsatisfiable: DoNotSchedule
          labelSelector:
            matchLabels:
              app: <фамилия>-app
```

В `k8s/postgres-deployment.yaml` закрепите БД за master:

```
    spec:
      nodeSelector:
        role: db
```

В `k8s/app-service.yaml` задайте свой NodePort:

```
spec:
  type: NodePort
  ports:
    - port: 8000
      targetPort: 8000
      nodePort: <ваш_nodeport>
```

В `app-configmap.yaml` поставьте `APP_OWNER: <ваша_фамилия>`, секреты оставьте в `app-secret.yaml` (настоящие пароли в публичный репозиторий не кладём, берём учебные вида `pgpass_<цифры>`).

Задание 3.2 - Примените манифесты

```
kubectl apply -f k8s/
kubectl get pods -o wide
```

В колонке `NODE` поды приложения должны быть и на master, и на worker.

Задание 3.3 - Проверьте доступ с обеих машин

```
curl http://<IP_master>:<ваш_nodeport>/
curl http://<IP_worker>:<ваш_nodeport>/
curl http://<IP_master>:<ваш_nodeport>/health
```

NodePort открыт на каждом узле кластера, даже если под запущен на другом.

Задание 3.4 - Покажите балансировку

```
for i in $(seq 1 10); do curl -s http://<IP_worker>:<ваш_
```

В выводе должны быть разные имена подов.

Задание 3.5 - Проверьте кэш и БД

```
curl http://<IP_master>:<ваш_nodeport>/weather/Moscow
curl http://<IP_worker>:<ваш_nodeport>/weather/Moscow
curl -X POST http://<IP_master>:<ваш_nodeport>/notes -H "
curl http://<IP_worker>:<ваш_nodeport>/notes
```

Первый запрос погоды - `"source": "api"`, второй - `"source": "cache"`, заметка читается через другой узел.

Этап 4. Простой CD: обновление приложения

Задание 4.1 - Измените приложение

Внесите любое видимое изменение (например, текст приветствия в `GET /`), сделайте commit и push в `lab-6`. Дождитесь зелёного пайплайна и скопируйте новый короткий SHA из Actions.

Задание 4.2 - Выполните rolling update

```
kubectl set image deployment/<фамилия>-app app=ghcr.io/<g
kubectl rollout status deployment/<фамилия>-app
kubectl get pods -o wide
```

Если имя контейнера в манифесте отличается, посмотрите его: `kubectl get deployment <фамилия>-app -o jsonpath='{.spec.template.spec.containers[*].name}'`.

Задание 4.3 - Проверьте результат и откат

```
curl http://<IP_master>:<ваш_nodeport>/
kubectl rollout history deployment/<фамилия>-app
kubectl rollout undo deployment/<фамилия>-app
kubectl rollout status deployment/<фамилия>-app
```

Вопрос: почему в манифесте для production лучше использовать тег с SHA (или digest), а не `latest`?

Бонус (+1 балл, необязательно): отразите обновление образа в самом репозитории - обновите тег в `k8s/app-deployment.yaml` отдельным коммитом и выполните `kubectl apply -f k8s/`, показав, что манифест в Git соответствует состоянию кластера.

Этап 5. Отказоустойчивость и финальный артефакт

Задание 5.1 - Отключите worker

На worker выполните `sudo systemctl stop k3s-agent` и через 1-2 минуты на master:

```
kubectl get nodes
kubectl get pods -o wide
```

Опишите в отчёте, что произошло с подами. Затем верните агент: `sudo systemctl start k3s-agent`.

Задание 5.2 - Финальный артефакт (на master)

```
echo "========================================" && \
echo "ЛР6 завершена: $(date '+%d.%m.%Y %H:%M:%S')" && \
echo "Студент: $(whoami)@$(hostname)" && \
echo "========================================" && \
kubectl get nodes -o wide && \
echo "" && \
kubectl get pods -o wide && \
echo "" && \
kubectl get svc && \
echo "" && \
kubectl get deployment <фамилия>-app -o jsonpath='{.spec.
```

Скриншот полного вывода. Должны быть видны:

1. Два узла `Ready` с разными IP
2. Поды приложения на обоих узлах
3. Ваш NodePort
4. Логин, хост и время выполнения
5. Образ с тегом-SHA

Оформление репозитория

Работаем в том же репозитории, ветка `lab-6`. Проект лежит в `solutions/assist-py/`. Все команды из этой работы выполняйте из каталога `solutions/assist-py/`. README оформляем по образцу из ЛР №2. Что должно появиться в репозитории:

- `.github/workflows/ci.yml`;
- в начале `README.md` бейдж статуса CI (в Actions: меню «Create status badge»);
- в `README.md` раздел «CI/CD и кластер»: схема узлов и их роли, как выкатить новую версию и как её откатить;
- паролей, токенов и `node-token` в репозитории нет;
- коммиты вида `lab6: add ci workflow`.

Ссылку на репозиторий добавьте в отчёт.

Критерии приема результатов

Этап	Баллы
1. Кластер из двух узлов	4
2. CI в GitHub Actions	4
3. Развёртывание на двух узлах	4
4. Простой CD (rolling update)	2
5. Отказоустойчивость и артефакт	1
Итого	15

-0-8-:7-I-B">

Требования к защите

1. Скриншот `kubectl get nodes -o wide`: два узла `Ready`.
2. Ссылка на успешный запуск GitHub Actions (оба job зелёные).
3. Скриншот `kubectl get pods -o wide`: поды приложения на двух узлах.
4. Ответы `GET /` с разными значениями `hostname` (имена подов Kubernetes).
5. `/weather/Moscow`: первый запрос `"source": "api"`, второй `"source": "cache"`.
6. Rolling update: вывод `kubectl rollout status` и новый ответ приложения.
7. `APP_OWNER` содержит вашу фамилию (через ConfigMap).

-;-<">

Типичные проблемы

Типичные проблемы и решения

Симптом	Причина и решение
`ping` между машинами не проходит	Изоляция клиентов в Wi-Fi или ВМ в режиме NAT. Используйте кабель, хотспот телефона или режим Bridged
Worker не появляется в `get nodes`	Закрыт порт 6443 или неверный токен/IP. Смотрите `sudo journalctl -u k3s-agent -f`
Узел `Ready`, но поды на разных узлах не видят друг друга	Закрыт UDP 8472 или выбран не тот сетевой интерфейс. Добавьте `--flannel-iface`
`exec format error` в логах пода	Архитектура образа не совпадает с узлом. Проверьте `platforms` в workflow

Формат сдачи

Файл: ЛР6_Фамилия_ИО_ИСУ.pdf (например, ЛР6_Иванов_ИА_408145.pdf).

Обязательные разделы:

1. Заполненная таблица персональных параметров.
2. Схема вашей сети: IP обоих узлов, способ подключения (кабель, Wi-Fi, ВМ в Bridged).
3. Скриншоты по пунктам «Требования к защите».
4. Полный текст `.github/workflows/ci.yml`.
5. Ссылка на репозиторий и на запуск GitHub Actions.
6. Ответы на вопросы из заданий 1.4, 4.3 и описание наблюдений из задания 5.1.